

Multi-threading

Advanced Programming
Brent van Bladel



Introduction

```
1. std::vector<int> data = importData();  
2.  
3. for (int& datapoint: data) {  
4.     double result = longCalculation(datapoint);  
5.     std::cout << result << std::endl;  
6. }
```

Why not perform the long calculations in parallel?

→ **Start a new thread for every datapoint !**

Multithreading

A **thread** is an execution context, i.e. all the information a CPU needs to execute a stream of instructions.

Multithreading is the process in which several threads are executed together.

Multithreading is implemented in the standard library.

```
#include<thread>
```

std::thread

```
1. #include <thread>
2.
3. int main() {
4.     std::thread t; // does not start a thread
5.     return 0;
6. }
```

How to provide the functionality that should be executed?

- Pointer to Function
- Lambda function

Pointer to Function

Similar to objects and variables, a function body is stored in memory. (static ROM memory)

→ **We can point to it!**

Pointer to Function

Syntax:

`return_type (* function_pointer_variable ) ( parameter_list )`

```
1. void hello() {  
2.     std::cout << "Hello world" << std::endl;  
3. }  
4.  
5. int main() {  
6.     void (*func_pointer)() = hello;  
7.     func_pointer();  
8.     return 0;  
9. }
```

Variable declaration of type
“Pointer to void-function without
parameters” (on the stack).

Assignment to function “hello”.

Call of pointed-to function.

Pointer to Function

```
1.  int sum(int a, int b) {  
2.      return a + b;  
3.  }  
4.  
5.  int main() {  
6.      int (*pointerToSum)(int, int);  
7.      pointerToSum = sum;  
8.  
9.      std::cout << pointerToSum(5, 8) << std::endl;  
10.     return 0;  
11. }
```

Pointer to Function

Note: For pointer assignment, the complete function type must match

```
1.  int foo(string s);  
2.  
3.  void bar(double d);  
4.  
5.  int main() {  
6.      int (*fp)(double);  
7.      fp = foo; // error: param type incorrect  
8.      fp = bar; // error: return type incorrect  
9.  }
```


std::thread - Pointer to Function Constructor

```
1. void parallelLoopBody(int x) {  
2.     double result = longCalculation(x);  
3.     std::cout << result << std::endl;  
4. }  
5.  
6. int main() {  
7.     std::vector<int> data = importData();  
8.  
9.     for (int& datapoint: data) {  
10.         std::thread th{parallelLoopBody, datapoint};  
11.         th.detach();  
12.     }  
13. }
```

Not a function you would want to expose.

→ **Solution:**

- Anonymous namespace
- Lambda Function

Lambda Functions

A **lambda function** is an anonymous function.

Useful for short snippets of code that are not going to be reused.

```
[ capture_clause ] (parameters) -> return_type  
{  
    definition_of_method  
}
```

```
[] (int a, int b) -> int { return a + b; }
```

Lambda Functions

Empty capture
clause, 1 integer
parameter, no
return type.

```
1. vector<int> v{1,2,3,4,5};  
2.  
3. for_each(v.begin(), v.end(),  
4. [] (int i) { cout << i << " "; }  
5. );
```

Function body

`std::for_each` applies a function to every element of a container
implemented in the standard library: `#include<algorithm>`

std::thread - Lambda Function Constructor

```
1.  int main() {  
2.      std::vector<int> data = importData();  
3.  
4.      for (int& datapoint: data) {  
5.          std::thread th{ [](int x) {  
6.              double result = longCalculation(x);  
7.              std::cout << result << std::endl;  
8.          } , datapoint};  
9.          th.detach();  
10.     }  
11. }
```

Lambda Functions - capture clause

A lambda function can capture variables from the enclosing scope in its capture clause by value or by reference.

- [&] : capture all external variables by reference
- [=] : capture all external variables by value

Alternatively, you can capture specific variables

- [&a, &b] : capture variables a and b by reference
- [a, b] : capture variables a and b by value
- [a, &b] : capture variables a by value and b by reference

Lambda Functions - capture clause

```
1. vector<int> v{1,2,3,4,5};
2. int increment = 5;
3. int total = 0;
4.
5. for_each(v.begin(), v.end(),
6.         [ increment, &total ] (int& i) {
7.             i += increment;
8.             total += i;
9.         }
10. );
```

for_each takes as arguments:

- `iterator<TElement>`
- `iterator<TElement>`
- `void(*) (TElement)`

Capture variable “increment” by value and variable “total” by reference.

std::thread - Lambda Function Constructor

```
1.  int main() {  
2.      std::vector<int> data = importData();  
3.  
4.      for (int& datapoint: data) {  
5.          std::thread th{ [datapoint]() {  
6.              double result = longCalculation(datapoint);  
7.              std::cout << result << std::endl;  
8.          }  
9.      };  
10.     th.detach();  
11. }  
12. }
```

Lambda Functions - Return

```
1.  vector<int> v{1,2,3,4,5};
2.
3.  if (all_of(v.begin(), v.end(), [](int i) -> bool {return i % 2 == 0;}))
4.      cout << "All numbers are even";
5.
6.  if (any_of(v.begin(), v.end(), [](int i) -> bool {return i % 2 == 0;}))
7.      cout << "Some numbers are even";
8.
9.  if (none_of(v.begin(), v.end(), [](int i) -> bool {return i % 2 == 0;}))
10.     cout << "All numbers are odd";
```

std::all_of, std::any_of, and std::none_of implemented in the standard library: #include<algorithm>

Lambda Functions - Function Pointer

```
1. vector<int> v{1,2,3,4,5};
2.
3. bool (*is_even)(int) = [](int i) -> bool {return i % 2 == 0;};
4.
5. if (all_of(v.begin(), v.end(), is_even))
6.     cout << "All numbers are even";
7.
8. if (any_of(v.begin(), v.end(), is_even))
9.     cout << "Some numbers are even";
10.
11. if (none_of(v.begin(), v.end(), is_even))
12.     cout << "All numbers are odd";
```

Lambda functions can also be stored as a function pointer*.

*if it does not capture anything

std::thread - Function Pointer to Lambda Function Constructor

```
1.  int main() {  
2.      std::vector<int> data = importData();  
3.  
4.      void (*loopBody)(int) = [](int x) {  
5.          double result = longCalculation(x);  
6.          std::cout << result << std::endl;  
7.      };  
8.  
9.      for (int& datapoint: data) {  
10.         std::thread th{loopbody, datapoint};  
11.         th.detach();  
12.     }  
13. }
```

std::thread - destructor

Destructing a thread that is still running will cause a runtime error.

```
1. int main() {  
2.     std::thread th{someLongCalculation}; // start thread th  
3.     return 0; // destructor called on th -> runtime error  
4. }
```

Solution: join or detach.

std::thread - detach

Calling detach on a thread separates that thread from the variable. The variable can then be safely destroyed.

```
1. int main() {  
2.     std::thread th{someLongCalculation}; // start thread th  
3.     th.detach(); // detach the thread from variable th  
4.     return 0; // destructor called on th -> OK!  
5. }
```

std::thread - join

Calling join on a thread waits for that thread to finish.

```
1. int main() {  
2.     std::thread th{someLongCalculation}; // start thread th  
3.  
4.     // do something else in the meantime  
5.  
6.     th.join(); // wait for the thread to be finished  
7.     return 0; // destructor called on th -> OK!  
8. }
```

Useful if you need the result of the thread calculation.

Promise and Future

A **promise/future** pattern solves the problem of transferring a value from one thread to another.

Implemented in the standard library: `#include<future>`

- Both promise and future objects share ownership of the value that is transferred.
- One thread is owner of the promise. The promise can set the value.
- The other thread is owner of the future. The future can get the value.

std::promise and std::future

```
1. void calculation(std::promise<int> && p) {  
2.     p.set_value(1);  
3. }  
4.  
5. int main() {  
6.     std::promise<int> p;  
7.     std::future<int> f = p.get_future();  
8.  
9.     // start thread th  
10.    std::thread th{&calculation, std::move(p)};  
11.  
12.    th.join(); // wait for calculation to finish  
13.    int result = f.get();  
14.    std::cout << result << std::endl;  
15. }
```

The promise of a value (int) that will need to be fulfilled.

Will contain a value (int) in the future, when the promise gets fulfilled.

Ask the thread to fulfill a promise.

Get the result which was promised (i.e. the future is now!)

Access Data from Multiple Threads

What if we need to access the same data from multiple threads?

→ **Possible, but dangerous!**

Data Race

A **data race** occurs when 2 instructions from different threads access the same memory location without synchronization, and at least one of the accesses is a write.

```
1.  int main() {  
2.      int count = 0;  
3.      std::thread t1{ [&]() { count++; } };  
4.      std::thread t2{ [&]() { count++; } };  
5.      t1.join();  
6.      t2.join();  
7.      std::cout << count << std::endl;  
8.  }
```

Data Race

```
1. int main() {  
2.     int count = 0;  
3.     std::thread t1{ [&]() { count++; } };  
4.     std::thread t2{ [&]() { count++; } };  
5.     t1.join();  
6.     t2.join();  
7.     std::cout << count << std::endl;  
8. }
```

Non-deterministic behaviour!

Possible scenario 1:

t1 executes completely

t2 executes

→ **count ends as 2**

Possible scenario 2:

t1 reads count as 0

t2 reads count as 0

t1 calculates 0 + 1

t2 calculates 0 + 1

t1 writes 1 to count

t2 writes 1 to count

→ **count ends as 1**

Race Condition

A **race condition** is a semantic error or bug that occurs when the timing or ordering of events affects a program's correctness.

Note: many race conditions are due to data races, and many data races lead to race conditions.

```
1.  int x = 1;
2.
3.  void reader() {
4.      if (x != 0)
5.          std::cout << 100/x << std::endl;
6.  }
7.
8.  void writer() { x = 0; }
9.
10. int main() {
11.     std::thread t1{reader};
12.     std::thread t2{writer};
13.     t1.join();
14.     t2.join();
15. }
```

Access Data from Multiple Threads

What if we need to access the same data from multiple threads?

→ **Possible, but dangerous!**

Solutions:

- **atomic:** solves data races
- **mutex:** solves race conditions

std::atomic

Objects of **atomic** types are the only C++ objects that are free from data races; that is, if one thread writes to an atomic object while another thread reads from it, the behavior is well-defined.

std::atomic is a wrapper that makes any supported type atomic

1. `int count = 0;`
2. `std::thread t1{ [&]() { count++; } };`
3. `std::thread t2{ [&]() { count++; } };`
4. `// -> undefined behaviour`

1. `std::atomic<int> count = 0;`
2. `std::thread t1{ [&]() { count++; } };`
3. `std::thread t2{ [&]() { count++; } };`
4. `// -> OK!`

std::atomic

Note that an atomic cannot solve every **race condition**.

→ **We need a mutex!**

```
1.  std::atomic<int> x = 1;
2.
3.  void reader() {
4.      if (x != 0)
5.          std::cout << 100/x << std::endl;
6.  }
7.
8.  void writer() { x = 0; }
9.
10. int main() {
11.     std::thread t1{reader};
12.     std::thread t2{writer};
13.     t1.join();
14.     t2.join();
15. }
16.
```

std::mutex

The mutex class is a synchronization primitive that can be used to protect shared data from being simultaneously accessed by multiple threads.

If another thread has already locked the mutex, a call to lock will block execution until the lock is acquired.

→ **New danger: blocking execution indefinitely**

```
1.  int x = 1;
2.  std::mutex x_mutex;
3.
4.  void reader() {
5.      x_mutex.lock();
6.      if (x != 0)
7.          std::cout << 100/x << std::endl;
8.      x_mutex.unlock();
9.  }
10.
11. void writer() {
12.     x_mutex.lock();
13.     x = 0;
14.     x_mutex.unlock();
15. }
```

Deadlocks

A **deadlock** is a situation where two threads are indefinitely waiting on each other to release a mutex.

```
1. void thread1 () {  
2.     mutex_1.lock();  
3.     // do something  
4.     mutex_2.lock();  
5.     // do something  
6.     mutex_2.unlock();  
7.     // do something  
8.     mutex_1.unlock();  
9. }
```

```
1. void thread2 () {  
2.     mutex_2.lock();  
3.     // do something  
4.     mutex_1.lock();  
5.     // do something  
6.     mutex_1.unlock();  
7.     // do something  
8.     mutex_2.unlock();  
9. }
```


Volatile Variables

A **volatile** object is one whose value might change spontaneously. When you declare an object to be volatile, you're telling the compiler that the object might change state even though no statements in the program appear to change it.

```
1.  int counter;
2.
3.  void thread1() {
4.      counter = 0;
5.      while (counter == 0)
6.          sleep(1); // sleeps indefinitely
7.  }
8.
9.  void thread2() { counter++; }
```

```
1.  volatile int counter;
2.
3.  void thread1() {
4.      counter = 0;
5.      while (counter == 0)
6.          sleep(1); // OK!
7.  }
8.
9.  void thread2() { counter++; }
```

Thread_local variables

A **thread_local** variable has a copy in each thread. When referring to it, the copy associated with the current thread is used.

```
1.  thread_local int x = 0;
2.
3.  void task() {
4.      x++;
5.      std::cout << x << std::endl;
6.  }
7.
8.  void thread2() {
9.      std::thread t1{task};
10.     std::thread t2{task};
11.     t1.join();
12.     t2.join();
13. }
```

Multi-threading

Quiz

What is the output of this code?

```
1. void print(int x) {  
2.     cout << x << endl;  
3. }  
4.  
5. int main() {  
6.     void (*p)(int) = print;  
7.     (*p)(5);  
8. }
```

Answer: “Result: 5”

A function pointer is a pointer, and thus can be dereferenced.

Since the pointer contains the address of a function, dereferencing it gives you the function.

Note: `(*p)(5)` is the same as writing `p(5)`

What is the output of this code?

```
1. void print(int x) {  
2.     cout << x << endl;  
3. }  
4.  
5. int main() {  
6.     void (*p)(int) = print;  
7.     void (*p2)(int) = p;  
8.     p2(5);  
9. }
```

Answer: “Result: 5”

The address stored in p is copied and saved in p2.

Note: p and p2 are stored on the stack as local variables, normal rules apply

What is the output of this code?

```
1. void print(int x) {  
2.     cout << x << endl;  
3. }  
4.  
5. int main() {  
6.     void (*p)(int) = print;  
7.     void (*p2)(int) = *p;  
8.     p2(5);  
9. }
```

Answer: “Result: 5”

The address stored in p is dereferenced, returning a function. P2 is initialized with this function.

Note: this is the same as initializing p2 with print directly

Introducing an Alias

```
1. void print(int x) {  
2.     cout << x << endl;  
3. }  
4.  
5. int main() {  
6.     using FuncInt = void (*)(int);  
7.     FuncInt p = print;  
8.     FuncInt p2 = *p;  
9.     p2(5);  
10. }
```

You can introduce an alias for complex types, such as function pointers.

→ **Generic Programming** (see chapter 10)

What is the output of this code?

```
1. int main() {  
2.     int *(*allocate)(int) = [](int x){ return new int{x}; };  
3.     cout << allocate(5) << endl;  
4. }
```

Answer: “0x563f38734eb0”

→ Function pointer to a function returning a pointer.

Also note the memory leak!

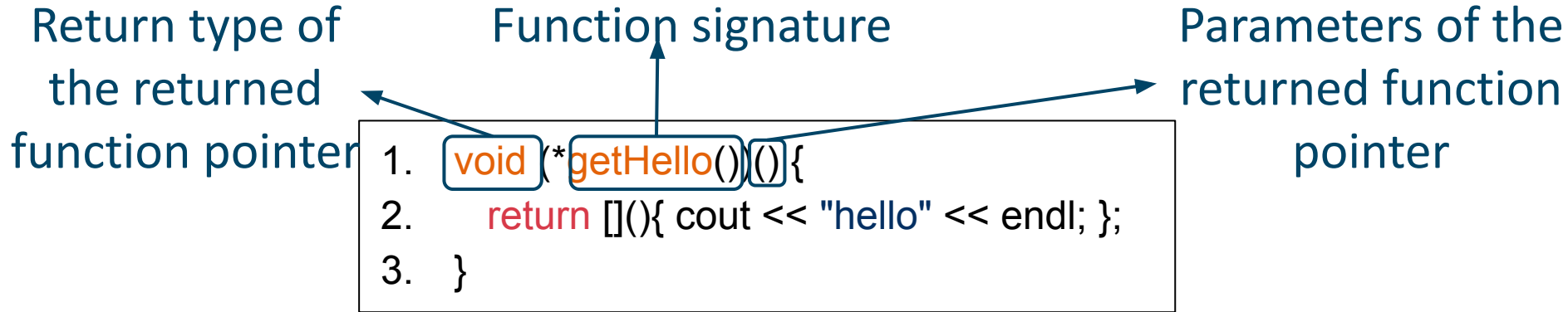
What is the output of this code?

```
1. void hello() {  
2.     cout << "Hello" << endl;  
3. }  
4.  
5. int main() {  
6.     void (*(*getHello())()) = [](){ return hello; };  
7.     getHello()();  
8. }
```

Answer: “Hello”

→ Function pointer to a function returning a function pointer.

Function returning a function pointer



Put a pointer in front of the signature, and brackets around that, to indicate returning a function pointer.

Introducing an Alias

```
1. using VoidFunc = void (*)();  
2.  
3. VoidFunc getHello() {  
4.     return [](){ cout << "hello" << endl; };  
5. }
```

What is the output of this code?

```
1.  int apply(int(*f)(int, int), int a, int b) {  
2.      return f(a,b);  
3.  }  
4.  
5.  int main(){  
6.      int (*sum)(int, int) = [](int x, int y){ return x + y; };  
7.      cout << apply(sum, 4, 6) << endl;  
8.  }
```

Answer: “10”

You can pass a function pointer as an argument to a function.

What is the output of this code?

```
1.  int main(){  
2.      int x = 5;  
3.      void (*print)() = [x]() { cout << x << endl; };  
4.      print();  
5.  }
```

Answer: Compilation error

You cannot assign a lambda with capture to a function pointer.

A lambda function is actually an object that behaves like a function. This object has a public conversion function to a function pointer if the capture clause is empty.

std::function

std::function is an object wrapper that can contain almost any object that acts like a function pointer.

```
1.  int main(){  
2.      int x = 5;  
3.      std::function<void()> print = [x]() { cout << x << endl; };  
4.      print();  
5.  }
```

std::function is implemented in the standard library.

```
#include<functional>
```

What is the output of this code?

```
1.  int main(){  
2.      int x = 4;  
3.      int y = 6;  
4.      std::function<void()> printSum =  
5.          [=]() { cout << x + y << endl; };  
6.      printSum();  
7.  }
```

Answer: "10"

What is the output of this code?

```
1.  int main(){
2.      int *x, *y; //declarations
3.      std::function<void()> printSum = [=]() { cout << (*x + *y) << endl; };
4.
5.      x = new int{4}; // calculation
6.      y = new int{6};
7.      printSum();
8.
9.      delete x; // clean-up
10.     delete y;
11. }
```

Answer: Undefined behavior, most likely a crash.

→ [=] captures everything by value at the moment of definition!

What is the output of this code?

```
1.  int main(){
2.      int *x, *y; //declarations
3.      std::function<void()> printSum = [&]() { cout << (*x + *y) << endl; };
4.
5.      x = new int{4}; // calculation
6.      y = new int{6};
7.      printSum();
8.
9.      delete x; // clean-up
10.     delete y;
11. }
```

Answer: “10”

What is the output of this code?

```
1.  int main(){  
2.      int x;  
3.      std::function<void()> print = [&x]() { cout << x << " "; };  
4.  
5.      for (x = 1; x <= 5; x++)  
6.          print();  
7.  }  
8.
```

Answer: "1 2 3 4 5 "

You can call the same lambda multiple times.

What is the output of this code?

```
1.  int main(){
2.      int x = 0;
3.      int y = 0;
4.      std::function<void()> printSums = [&]() {
5.          std::function<int()> sum = [=]() { return x + y; };
6.          for (x = 0; x <= y; x++)
7.              cout << sum() << " ";
8.      };
9.      y = 5;
10.     printSums();
11. }
```

Answer: “5 5 5 5 5”

The inner lambda captures by value at the moment the outer lambda is executed!

What is the output of this code?

```
1.  int main(){  
2.      int x = 0;  
3.      thread th([&x]() {  
4.          while (x < 100)  
5.              cout << x++ << endl;  
6.      });  
7.      th.detach();  
8.  }
```

Answer: Undefined behaviour, most likely a crash.

Thread th captures x by reference and is detached. As soon as the main thread ends, x becomes a dangling reference in thread th.

What is the output of this code?

```
1.  int main(){
2.      int x = 0;
3.      thread th1([&x](){ x++; cout << "th1: " << x << endl; });
4.      thread th2([&x](){ x++; cout << "th2: " << x << endl; });
5.      th1.join();
6.      th2.join();
7.  }
```

Answer: Undefined behaviour due to data race!

Variable x can be either 1 or 2 in the end.

What is the output of this code?

```
1.  int main(){
2.      atomic<int> x = 0;
3.      thread th1([&x]() { x++; cout << "th1: " << x << endl; });
4.      thread th2([&x]() { x++; cout << "th2: " << x << endl; });
5.      th1.join();
6.      th2.join();
7.  }
```

Answer: Undefined behaviour due to race condition!

Variable x will always be 2 in the end, but the output is not guaranteed to be correct.

Example: t1 and t2 could first both calculate x++, and then both print that x is 2.

What is the output of this code?

```
1. int main(){
2.     int x = 0;
3.     mutex mx;
4.     thread th1([&](){ mx.lock(); x++; cout << "th1: " << x << endl; mx.unlock();});
5.     thread th2([&](){ mx.lock(); x++; cout << "th2: " << x << endl; mx.unlock();});
6.     th1.join();
7.     th2.join();
8. }
```

Answer: Non-deterministic: either “t1: 1 or t2: 1
 t2: 2” t1: 2”

What is the output of this code?

```
1. int main(){  
2.     thread th1([](){ cout << "Hello" << "World" << endl; });  
3.     thread th2([](){ cout << "Hello" << "World" << endl; });  
4.     th1.join();  
5.     th2.join();  
6. }
```

Answer: Non-deterministic!

For example: “HelloHelloWorldWorld”

std::syncstream (c++20)

A **syncstream** is a wrapper around a stream that provides synchronization between threads writing to the same stream.

```
1.  int main(){
2.      thread th1([](){
3.          std::osyncstream out{cout};
4.          out << "Hello" << "World" << endl;});
5.      thread th2([](){
6.          std::osyncstream out{cout};
7.          out << "Hello" << "World" << endl;});
8.      th1.join();
9.      th2.join();
10. }
```